hw

1.Schema representation of star type.

a) Schema (Database Schema)

➡️ Logical structure that defines how data is organized in a database:

- Tables
- Relationships
- Constraints

Example: In your case, Rain Prediction Schema defines tables like `FactWeather`, `DimLocation`, etc.

---

b) Star Schema Structure

➡️ A **type of database schema** optimized for analytical queries:

- **Central Fact Table** with measurable facts (e.g., weather data)
- Multiple **Dimension Tables** providing context (e.g., location, date)
- Visually resembles a star

---

c) Fact Table

➡️ Stores transactional or measurable event data

➡️ Contains **foreign keys** linking to dimensions

Example in Your Dataset:

`FactWeather` with measurements like rainfall, temperature, wind speed

---

d) Dimension Table

➡️ Descriptive tables providing context for facts

➡️ Typically denormalized for fast querying

Examples in Your Dataset:

- `DimLocation`: City, state, country

- `DimDate`: Day, month, year
- `DimWindDirection`: Wind directions (N, S, NE, etc.)

Star Schema Representation (Text Diagram)

```
[DimDate]
   |
[FactWeather]-----[DimLocation]
   |
[DimWindDirection]
```

Table Roles:

Table Name	Role
`FactWeather`	Central Fact Table
`DimDate`	Date Dimension
`DimLocation`	Location Dimension
`DimWindDirection`	Wind Direction Dimension

Fact Weather

Column Name	Data Type
WeatherID (PK)	INT
DateID (FK)	INT
LocationID (FK)	INT
WindGustDirID (FK)	INT
WindDir9amID (FK)	INT
WindDir3pmID (FK)	INT
MinTemp	FLOAT
MaxTemp	FLOAT

Rainfall	FLOAT
Evaporation	FLOAT
Sunshine	FLOAT
WindGustSpeed	FLOAT
WindSpeed9am	FLOAT
WindSpeed3pm	FLOAT
Humidity9am	FLOAT
Humidity3pm	FLOAT
Pressure9am	FLOAT
Pressure3pm	FLOAT
Cloud9am	FLOAT
Cloud3pm	FLOAT
Temp9am	FLOAT
Temp3pm	FLOAT
RainToday	ENUM('Yes','No')
RainTomorrow	ENUM('Yes','No')

Dim Date (Date Dimension Table)

Column Name	Data Type
Date ID (PK)	INT
Full Date	DATE
Day	INT
Month	INT
Year	INT
Weekday	VARCHAR(10)

Dim Location (Location Dimension Table)

Column Name	Data Type
LocationID (PK)	INT
LocationName	VARCHAR(50)
Country	VARCHAR(50)

State	VARCHAR(50)
City	VARCHAR(50)

Dim Wind Direction (Wind Direction Dimension Table)

Column Name	Data Type
DirectionID (PK)	INT
Direction	VARCHAR(5)

Uses of Star Schema

✅ Best for **Data Warehousing** and **Business Intelligence (BI)**

✅ Optimized for **fast, complex queries**, like:

- "Rainfall trends by location and year"
- "Impact of wind direction on rain prediction"
- "Average temperatures by city for specific months"

✅ Simplifies slicing data by time, geography, direction

✅ Reduces JOIN complexity compared to normalized OLTP systems

✅ Common in tools like Power BI, Tableau, Data Analytics platforms

Snowflake Schema, Galaxy Schema and Data Vault Scheme overview

Snowflake Schema Overview

Definition:

A **normalized** extension of the Star Schema where Dimension Tables are further broken down into related sub-tables to reduce redundancy.

Visual:

Looks like a "snowflake" with branching dimension structures.

Structure Example (Rain Dataset Context):

```
less
CopyEdit
          [DimCountry]   [DimState]
               \         /
             [DimLocation]
                  |
              [FactWeather]
                  |
            [DimWindDirection]
```

Key Characteristics:

✅ Dimension tables normalized into sub-tables (removes duplicates)

✅ Uses foreign key relationships for hierarchical breakdowns

✅ More complex joins than Star Schema

✅ Saves storage space

Example:

Instead of storing country, state, city repeatedly in `DimLocation`, they exist in separate tables:

- `DimCountry`, `DimState`, `DimCity`

When to Use:

- Data is large, high redundancy expected
- Storage efficiency prioritized
- Read performance slightly sacrificed for space

Galaxy Schema (Fact Constellation Schema)

Definition:

Multiple Fact Tables share common Dimension Tables, creating a "constellation" of facts and dimensions.

Structure Example (Rain Dataset Expanded):

```
less
CopyEdit
            [DimLocation]      [DimDate]       [DimWindDirection]
              |                |                |
    [FactWeather]            [FactRainfall]       [FactWindSpeed]
```

Key Characteristics:

✅ Combines multiple fact tables for different business processes

✅ Dimensions are shared across facts

✅ Scalable and supports complex reporting

Example in Weather Context:

- `FactWeather`: General weather data
- `FactRainfall`: Detailed rainfall events
- `FactWindSpeed`: High wind measurements

All facts can use `DimLocation`, `DimDate`, etc.

When to Use:

- Large enterprises with multiple analytical areas
- Reporting requires cross-comparison of different processes
- Complex, scalable analytical environments

Data Vault Schema

Definition:

A highly scalable, audit-friendly, and agile data model often used in modern enterprise data warehouses.

Structure Breakdown:

Component	Description
Hub	Core business keys (e.g., Location, Date)
Link	Relationships between Hubs
Satellite	Descriptive attributes, history, context

Visual Example (Rain Dataset):

```
less
CopyEdit
 [HubLocation]        [HubDate]
      \              /
        [LinkWeatherObservation]
              |
        [SatelliteWeatherDetails]
```

Key Characteristics:

✅ Separates business keys (Hubs) from context (Satellites)

✅ Links capture relationships

✅ Built for auditability and historical tracking

✅ Supports agile data evolution without breaking model

Example in Weather:

- `HubLocation`: Unique locations
- `HubDate`: Unique dates
- `LinkWeatherObservation`: Connects observations to location and date
- `SatelliteWeatherDetails`: MinTemp, MaxTemp, Rainfall, etc.

When to Use:

- Large, evolving data landscapes
- Need for historical data tracking (Time Travel)
- Regulatory or audit requirements
- Agile data development required

🎯 Quick Comparison Table

Aspect	Star Schema	Snowflake Schema	Galaxy Schema	Data Vault
Simplicity	High	Moderate	Moderate	Complex
Storage Use	More space	Space-efficient	Varies	Efficient
Query Speed	Faster	Slightly slower	Complex depends	Flexible
Flexibility	Moderate	Moderate	High	Very High
Best For	Simple BI	Large structured	Multi-area BI	Enterprise-grade, audit-driven

Data Lake, Data Lakehouse, Delta Lake vs Delta Live Tables

1️⃣ Data Lake

Definition:

A **Data Lake** is a centralized storage repository that holds vast amounts of raw, structured, semi-structured, and unstructured data at any scale.

Key Features:

✅ Stores data in its native format (CSV, JSON, Parquet, images, videos, logs)

✅ Supports large-scale storage, often in **cloud platforms**

✅ Ideal for both current use and future unknown use cases

Examples:

- **AWS S3 (Amazon Simple Storage Service)**
- **Azure Blob Storage**
- **Google Cloud Storage (GCS)**

Benefits:

✔️ Cost-effective, scalable storage

✔️ Supports machine learning, big data, and analytics

✔️ Handles structured (tables), semi-structured (JSON, XML), unstructured (images, video) data

---

Challenges:

⚠️ Lack of structure can lead to "Data Swamps" (unmanaged, messy data)

⚠️ Querying raw data may require significant processing

2️⃣ Data Lakehouse

Definition:

A **Data Lakehouse** combines features of a Data Lake (cheap, scalable storage) with a **Data Warehouse** (structured, fast analytics).

➡️ Supports raw data storage like a lake

➡️ Adds **schema, ACID transactions, governance, and performance** like a warehouse

---

Popular Example Technologies:

✅ **Delta Lake** (by Databricks)

✅ Apache Hudi, Apache Iceberg

✅ Databricks Lakehouse Platform

Why Data Lakehouse Exists:

- Data Lakes = flexible, but slow for structured queries
- Data Warehouses = fast, but expensive, structured-only

Lakehouse = Best of both worlds

Lakehouse Advantages:

✔️ ACID transactions (safe updates)

✔️ Schema enforcement

✔️ Supports both real-time and batch processing

✔️ Suitable for BI tools, ML pipelines, data science

3️⃣ Delta Lake vs. Delta Live Tables

Delta Lake

➡️ Open-source storage layer for building reliable Data Lakes with ACID transactions, version control, and performance boosts.

Features:

✅ ACID Transactions — Ensures consistency even during concurrent operations

✅ Schema enforcement — Prevents bad data from corrupting tables

✅ Time Travel — Query historical data versions

✅ Scalable — Built on Parquet files, works with Spark

Use Cases:

✔️ Data lakes needing structure and reliability

✔️ Batch and streaming pipelines

✔️ ML and BI-ready data storage

Delta Live Tables (DLT)

➡️ A **managed service by Databricks** that automates pipeline creation, monitoring, and data quality enforcement using Delta Lake.

Features:

✅ Built on Delta Lake

✅ Simplifies data pipeline development with SQL or Python

✅ Automates:

- Data freshness checks
- Pipeline monitoring
- Quality validations (e.g., "expect column A > 0")

  ✅ Supports batch and streaming pipelines

  ✅ Handles dependency tracking between tables

Delta Lake vs Delta Live Tables - Comparison

Feature	Delta Lake	Delta Live Tables
Type	Open-source storage layer	Managed pipeline service
Functionality	Provides ACID tables	Automates building pipelines
Data Handling	Manual control via Spark	Declarative pipeline coding
Pipeline Management	Developer's responsibility	Databricks handles orchestration
Ideal For	Building reliable lakes	Simplifying production pipelines

---

4️⃣ Summary

Term	Purpose
Data Lake	Cheap, scalable raw data storage (e.g., S3)
Data Lakehouse	Adds warehouse-like structure to Data Lakes
Delta Lake	Storage layer for ACID, reliable lakes
Delta Live Tables	Managed, automated pipeline service on Delta Lake